



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Facultat d'Informàtica de Barcelona



Inteligencia Artificial

Práctica 1: Búsqueda local



Jorge de la Fuente Martínez
Pol García Vernet
Roger Catalan Yarza

índice

Identificación del problema y consideraciones del enunciado.....	2
Estado del problema y representación.....	5
Descripción de la representación.....	5
Espacio de búsqueda.....	6
Representación y análisis de los operadores.....	8
Motivos de elección de operadores.....	8
Operadores utilizados.....	8
Factor de ramificación de los operadores.....	9
Análisis de la función heurística.....	10
Planteamiento de las heurísticas.....	10
Funciones heurísticas escogidas.....	11
Elección y generación del estado inicial.....	12
Predicción sobre los posibles resultados.....	13
Experimentación y resultados.....	15
Influencia de los operadores.....	15
Influencia de la solución inicial.....	17
Influencia de Parámetros de Simulated Annealing.....	21
Influencia del tamaño del problema con relación 5:100.....	22
Influencia del tamaño del problema con valores (+5, +50).....	23
Influencia del incremento de números de helicópteros disponibles.....	25
Diferencia entre las dos heurísticas.....	26
Conclusiones.....	29
Competencia transversal: Trabajo de Innovación.....	30

Identificación del problema y consideraciones del enunciado

En esta práctica se presenta un planteamiento que puede transcurrir en la vida real, relacionada con la gestión de recursos.

En un escenario cuadrado de 50 kilómetros de costado existen C centros de rescate, situados en el perímetro del escenario. Cada centro dispone de H helicópteros, siendo H un número natural igual para todos los centros. Estos helicópteros deben rescatar una cantidad G de grupos, que se pueden categorizar como prioritarios (de tipo 1, si tienen heridos) o no prioritarios (de tipo 2). Para ello, debemos asignar a los diferentes helicópteros cuáles son sus grupos a rescatar. Deben considerarse una serie de restricciones, como que un helicóptero no puede rescatar a más de tres grupos ni a más de 15 personas en un sólo viaje. Un grupo está compuesto entre una y doce personas, y debe ser rescatado al completo por un solo helicóptero en un sólo viaje. Los helicópteros aterrizan justo en las coordenadas donde está situado el grupo, y todos los desplazamientos se hacen a una velocidad constante de 100 km/h, siguiendo la distancia euclídea.

La duración de un viaje de un helicóptero debe, además de la duración del recorrido, sumar el tiempo de rescate del grupo: 2 minutos por cada miembro de un grupo prioritario, o un minuto por cada miembro de un grupo no prioritario. Cuando un helicóptero regresa a su centro, hay que añadir un tiempo de 10 minutos. En la vida real, se podría considerar que es un tiempo encargado en la recarga de combustible, provisiones y relevos. El tiempo total se calcula como la suma de los viajes de todos los helicópteros, ya que no se está considerando que los viajes se realizan de forma secuencial, y no en paralelo.

El objetivo de este problema es presentar una solución en la que, por cada helicóptero, se indique con qué orden ha rescatado a los grupos. Una solución es válida si, aparte de las restricciones ya indicadas, se rescatan a todos los grupos.

El enunciado de esta práctica es, evidentemente, una simplificación para poder realizar unos códigos y experimentos relacionados con algoritmos de búsqueda local. El motivo de por qué lo es se comentará más adelante. Teniendo en cuenta dicha simplificación, se debe puntualizar una serie de consideraciones.

La representación del tablero no se ha expresado con unidades de menor magnitud por diversas cuestiones. Una de estas es que obtener dichas distancias en milímetros, por ejemplo, es tanto absurdo como inasumible. Un símil para clarificar mejor es en la red de carreteras, donde existen pequeños carteles para indicar el nombre de la vía y la distancia en kilómetros desde uno de los dos puntos de origen, que es el tomado como referente en los dos sentidos. Una de las utilidades que tiene es la de avisar con mayor eficacia a los servicios de emergencia. El otro motivo por el que no tiene sentido es por la elevación del coste computacional.

En las coordenadas es conveniente usar números naturales en lugar de reales. ¿Por qué? Básicamente, porque el conjunto de los reales es denso. En otras palabras, entre dos números reales siempre existen infinitos números reales. Entre un real y otro, siempre existirá un real entre ambos números. Esa propiedad implica que no existe una función biyectiva a la que un número real se le pueda asignar un natural, representando su orden en una secuencia, a diferencia de con los números naturales, enteros o racionales. Por tanto, representar números reales mediante una lista de Java, por ejemplo, no es una buena idea.

El cálculo del tiempo por cada viaje también es una simplificación para tener un modelo de cálculo asumible. Para ello, se ha utilizado la fórmula básica de la velocidad.

$$v = \frac{\Delta d}{\Delta t}$$

Como la velocidad en el ejercicio es constante (100 km/h) y la distancia recorrida entre dos puntos es la euclidiana (otra simplificación por parte del enunciado, ya que trabajamos en un plano de dos dimensiones), el tiempo se calcula fácilmente de esta manera:

$$\Delta t = \frac{\Delta d}{v}$$

Como se puede intuir, se supone que, en todo momento, la velocidad siempre será la misma, siendo cero el valor de la aceleración. Eso no tiene sentido ya que se producen cambios de velocidad en el despegue, en que la velocidad inicial es cero y el helicóptero no podrá moverse si no se incrementa dicha velocidad. También se deja de lado otros factores como la aceleración de la gravedad, la curvatura de la Tierra, la masa o las diferentes fuerzas que pueden interactuar con el helicóptero. Sin embargo, eso aumentaría la complejidad del ejercicio y no es el objetivo de esta práctica. Esto no es un ensayo de física, sino un escenario sencillo en que se puedan llevar a cabo algoritmos de búsqueda local.

Se ha repetido en varias ocasiones que se van a utilizar algoritmos de búsqueda local. Más allá de que el título de la práctica ya lo induce, ¿por qué se tiene que hacer así? Un aspecto crucial es el ahorro computacional, tanto en memoria como en tiempo de ejecución. En este caso, no nos interesa hallar todas las probabilidades posibles y devolver la solución más óptima, sino obtener rápidamente una solución posible, aunque sea subóptima.

La pregunta consiste en hallar una respuesta válida, no la mejor de todas, lo que ahorra recorrer todas las combinaciones posibles. Otro aspecto a considerar es que, como no estamos considerando que los viajes se realizan en paralelo, sólo nos interesa la lista ordenada de los grupos que rescata cada helicóptero. A partir de ese orden ya se puede saber el número de viajes y los grupos a rescatar en cada viaje, ya que en cada uno se puede rescatar a un máximo de tres grupos y a un máximo de 15 personas. Por tanto, no hace falta guardar el camino entero realizado por los helicópteros.

Estado del problema y representación

Descripción de la representación

Para representar el estado del problema, se ha creado una clase en Java llamada *DesastresBoard*, en la que también existen los operadores de cambio de estado.

Uno de los planteamientos iniciales fue representar el escenario como una matriz de 50 filas y 50 columnas en que cada casilla tuviera dos campos: uno que indicase el tipo de elemento (casilla vacía, centro, grupo) y otra que guardase, si no está vacía, un identificador del centro o del grupo. Después, se concluyó que dificulta enormemente la implementación, ya que en para una misma coordenada (x,y) puede haber más de un grupo o centro, o incluso un grupo y un centro en una misma casilla. Así pues, hacer una matriz estática de *ArrayList* en Java, donde cada elemento sería una clase *Tipo* en que se indique el tipo y (si es necesario) un identificador, más allá del desperdicio de memoria por todas las casillas sin información relevante, no era una buena idea.

Al final, se ha decidido que la representación del estado mediante un *array* estático de helicópteros. Como el número de helicópteros por cada centro es la misma (H) y el número de centros (C) también es constante, se puede representar de la siguiente manera:

```
private Helicoptero[] representacionEstado;
```

El número de elementos siempre será el número total de helicópteros ($H \cdot C$), frente a los 2500 elementos (50x50) de la anterior forma, ahorrando memoria. La clase *Helicoptero* es privada y contiene los grupos que han sido rescatados en orden.

```
private class Helicoptero {
    public int idCentro;
    public ArrayList<Integer> gruposARescatar;

    // Aquí están las constructoras.
}
```

El campo *idCentro* se incluye únicamente por una cuestión práctica, ya que se puede saber directamente a qué centro pertenece un helicóptero en específico. Cada elemento *j* del

conjunto de los identificadores de un centro es un natural situado entre 0 y C-1. Expresado de forma matemática:

$$j \in [0, C - 1] \cap \mathbb{N}$$

Como H es constante, y el identificador de un helicóptero i es un natural situado entre 0 y H*C, el identificador de un centro a cada helicóptero se representa en la estructura de la clase *DesastresBoard* así, dando un natural como resultado:

```
this.representacionEstado[i].idCentro = i / H;
```

Este valor también se podría calcular en tiempo real cada vez que se necesite, pero no supone un aumento notorio de la memoria al tratarse de un entero, un tipo primitivo en Java.

Como no sabemos cuántos grupos va a terminar rescatando cada helicóptero (incluso se podría llegar a no rescatar ninguno), el campo *gruposARescatar* no puede ser una estructura estática de datos. Como los centros van a ser los mismos, y las coordenadas no varían a lo largo del ejercicio, simplemente basta con guardar un natural que lo identifique.

Para acceder a los grupos y a los centros, la propia constructora *DesastresBoard* ya pasa los datos, que van a ser guardados en estos campos:

```
private Centros centros;  
private Grupos grupos;
```

Estas dos clases ya vienen implementadas en el *Desastres.jar*, librería ofrecida junto a las instrucciones de la práctica.

Espacio de búsqueda

Los operadores y las heurísticas planteados son útiles para generar estados sucesores. Pero la pregunta es: ¿cuáles son los algoritmos? En la práctica se pide utilizar dos: *Hill Climbing* y *Simulated Annealing*. En ambos casos, el espacio de búsqueda es el conjunto de las soluciones posibles de un problema. Y la solución ofrecida será subóptima, porque no recorre todas las combinaciones posibles, pero será rápida de calcular.

Esta decisión de usar herramientas de búsqueda local responde a la inviabilidad de explorar todo el espectro de soluciones posibles, ya que al haber tantas variables, el resultado de todas sus combinaciones generaría tantas posibilidades que el tiempo de ejecución se volvería extremadamente alto para problemas de tamaño grande. Solo con imaginar algunas soluciones con criterio válido y sus posibles alternativas similares, es fácil darse cuenta de ello: un helicóptero del centro Cx rescata todos los grupos G, un helicóptero del centro Cx rescata G-1 grupos y un helicóptero del centro Cy rescata 1 grupo, etcétera. El resultado de la combinatoria resulta evidente escala exponencialmente, confirmando la inviabilidad.

En el caso del *Hill Climbing*, sólo se traslada de un estado a otro si el estado vecino encontrado mejora la solución. Su filosofía está basada en los algoritmos voraces (*Greedy*); es decir, la solución se va construyendo formando una sucesión de estados, de tal forma que el algoritmo no da marcha atrás, sino que sigue construyendo a partir de las decisiones anteriores. La heurística se encarga de rechazar todos los sucesores que sean inmediatamente peores (que no tiene por qué serlo al construirse una solución completa), lo que genera que el espacio de exploración sea solamente local. Esto se verá en los experimentos, pero esto justifica por qué el número de nodos explorados es bastante menor que usando *Simulated Annealing*.

En el caso del *Simulated Annealing*, el espacio de soluciones es el mismo, pero la exploración es global. Esto es debido a que puede aceptar soluciones peores, que varían en función de una probabilidad y una “temperatura”. Aunque, de primeras, pueda parecer peor, la ventaja reside en que este método permite escapar de óptimos locales, que no necesariamente pueden conducir a un óptimo global.

Representación y análisis de los operadores

Motivos de elección de operadores

La elección de operadores para generar nuevos estados es importante. Dichos operadores deben generar nuevas soluciones que sigan cumpliendo con las restricciones del enunciado y proporcionar soluciones válidas.

La elección de estos operadores se ha decidido considerando el planteamiento del problema y la representación de estado que hemos decidido. Debido a que en nuestra propuesta asignamos listas de grupos a helicópteros, las opciones que tenemos deben modificar dichas asignaciones. Al ser un problema de búsqueda local, debemos partir de una propuesta de solución completa, donde todos los grupos están asignados de alguna manera, en nuestro caso, mediante asignaciones aleatorias o asignando los grupos a helicópteros del centro más cercano a ellos.

Si hubiéramos planteado el problema de otra manera, por ejemplo creando sectores, podríamos disponer de otros operadores distintos, como el cambio de sector entre helicópteros.

Operadores utilizados

Para la generación de sucesores en la búsqueda local, se han implementado dos opciones de operadores:

- **Intercambio de grupos entre helicópteros**, donde un grupo G1 de un helicóptero H1 se cambia por otro grupo G2 del helicóptero H2. El intercambio de grupos permite la posibilidad de explorar nuevos recorridos de los helicópteros implicados, pudiendo, en un caso favorecedor, mejorar la solución al intercambiarse grupos que estén próximos a otros grupos de la asignación de cada helicóptero, maximizando la capacidad del helicóptero.
- **Cambio del orden de los grupos** para su rescate de cada helicóptero. Cada helicóptero dispone de una lista de grupos a rescatar, y el orden se modifica para que las nuevas propuestas de solución dispongan de diferentes recorridos. La heurística, de manera implícita, evalúa el estado maximizando el uso de la capacidad del helicóptero para minimizar el número de viajes, pero no considera la proximidad entre grupos. Por este motivo, un orden distinto donde los grupos presenten una combinación de tamaño de los grupos y proximidad entre ellos evaluará de manera favorable el estado.

Factor de ramificación de los operadores

El operador de **intercambio de grupos** entre helicópteros, nuestro operador tiene un factor de ramificación de orden $O(H^2)$, puesto que se hace un único intercambio por cada pareja posible de helicópteros (con la optimización de no realizar los emparejamientos simétricos), escogiendo el grupo más cercano al centro de la lista del segundo helicóptero de la pareja.

El operador de **cambio de orden** tiene un factor de ramificación mayor al intercambio de grupos, de orden $O(H \cdot G^2)$, puesto que generamos todas las combinaciones de cambio de orden para cada helicóptero.

Análisis de la función heurística

Planteamiento de las heurísticas

La función heurística utilizada para ir descartando otras soluciones es el tiempo empleado. El tiempo es una buena magnitud para ello, ya que el mejor valor corresponde al menor tiempo de viaje y, además, el tiempo nunca va a ser un número negativo.

Ya se ha explicado anteriormente en las simplificaciones acudidas en este problema, para obtener soluciones de una forma más sencilla. Así pues, como el tiempo total empleado por los helicópteros se considera de forma secuencial y la velocidad de los helicópteros es la misma, la fórmula final es la siguiente:

$$t = \frac{\Delta d}{v} + t_{descanso} + t_{rescatePrioritarioe} + t_{rescateNoPrioritarios}$$

Como el resultado se va a representar en minutos, los helicópteros toman siempre 10 minutos de descanso por cada viaje, todos van a 100 km/h, el tiempo de rescate de una persona de un grupo prioritario es de dos minutos y una de un grupo no prioritario es de un minuto, esta fórmula puede poseer más información:

$$t = \frac{\Delta d}{100/60} + 10 \cdot viajes + 2 \cdot PersonasGruposPrio + PersonasGruposNoPrio$$

Teniendo en cuenta que no se pueden rescatar más de 3 grupos, ni más de 15 personas, tal como está implementado, un helicóptero no vuelve a la base hasta que no pueda recoger a ningún otro grupo sin ser rescatado, respetando las restricciones.

Una vez identificada la fórmula para evaluar un estado de solución, es importante matizar que la heurística debe variar su comportamiento de evaluación en función de los criterios del enunciado. Para el criterio de minimización del tiempo total, no es necesario hacer ninguna modificación que considere la priorización de grupos. Sin embargo, para el criterio de minimizar el tiempo de rescate de grupos prioritarios, la heurística deberá primero considerar dichos grupos, optimizar su tiempo de rescate, y después realizar el mismo procedimiento con los grupos no prioritarios.

Funciones heurísticas escogidas

Si bien la evaluación heurística es esencialmente la misma, su implementación la hemos dividido en dos funciones diferentes para poder cumplir con los requisitos del enunciado. La diferencia recae en considerar, o no, la prioridad de cada uno de los grupos a rescatar para cada helicóptero.

En la heurística la hemos implementado con la idea de minimizar los viajes de los diferentes helicópteros.

Para cada helicóptero, analizamos su lista de grupos a rescatar. Buscando por orden dentro de la lista, analizamos si un grupo ha sido rescatado o no. En caso de no estarlo, consultamos el tamaño de este. Si cabe en el helicóptero, se rescata al grupo. En caso contrario, pasamos a analizar el siguiente grupo de la lista. Una vez rescatado un grupo, se comprueban las restricciones, como haber rescatado ya tres grupos o que se alcance la capacidad máxima. También consideramos el caso en que se haya hecho un análisis completo de la lista de grupos y no exista posibilidad de rescatar otro grupo, ya sea por tamaño de este o porque no queden más grupos que rescatar.

La heurística para priorizar el rescate de grupos prioritarios funciona de la misma manera, pero realizando el cálculo en dos fases. Una primera ronda considerando solo los grupos prioritarios y una segunda con los no prioritarios.

Al plantear la heurística de esta manera, generamos sinergia con la generación de nodos sucesores, ya que al aplicar los operadores, podemos llegar a encontrar mejores soluciones que optimicen tanto el tamaño del grupo como la distancia entre grupos, reduciendo aún más el número de viajes y el tiempo requerido para recorrer la distancia.

Elección y generación del estado inicial

Para la resolución de este problema, hemos decidido basarnos en dos ideas para generar estados iniciales, que se han traducido en un total de cuatro métodos para generar el estado inicial, ya que la restricción de minimizar el tiempo de rescate de grupos prioritarios nos genera un planteamiento inicial distinto. Los cuatro métodos que hemos escogido son:

- **Asignación aleatoria:** El objetivo de este método de generación inicial es la rapidez. Dada la lista de grupos G , asignamos cada grupo a un helicóptero de un centro aleatorio, sin considerar ningún criterio. Esta asignación no considera ningún tipo de optimización, ni referente a proximidad de grupos a centros y otros grupos, ni la prioridad de los grupos, ni de carga de trabajo entre los helicópteros. El coste de esta operación es del orden $O(G)$, pues debemos asignar todos los grupos G .
- **Asignación aleatoria prioritaria:** Sigue la misma filosofía que el método de asignación aleatoria, pero realizando un pequeño trabajo extra para asignar los grupos prioritarios primero y luego los no prioritarios. Este método está pensado para utilizarse con heurística prioritaria, donde dicha heurística se aprovechará de este esfuerzo. El coste de esta operación se mantiene en orden $O(G)$, pues deberemos hacer dos pasadas, una primera para separar los grupos prioritarios de los no prioritarios, y una segunda para asignarlos.
- **Asignación por distancias:** Este método asigna a cada grupo a un helicóptero de su centro más cercano, independientemente de su prioridad, balanceando la carga entre los helicópteros disponibles en dicho centro. No hay ninguna optimización referente a cargas entre centros, por lo que en un caso extremo donde todos los grupos tuviesen el mismo centro como más cercano, el resto de centros inicialmente no realizaría ningún trabajo. El coste de esta operación es del orden $O(G \cdot C)$, pues debemos, para cada grupo, navegar sus combinaciones con todos los centros disponibles.
- **Asignación prioritaria por distancias:** Igual que para la asignación aleatoria prioritaria, este método pretende asignar los grupos prioritarios a sus respectivos centros más cercanos primero, y luego procede a asignar los no prioritarios. El coste de esta operación se mantiene en el orden $O(G \cdot C)$, como en su versión sin considerar la prioridad.

Estos cuatro métodos deberían permitirnos ver el impacto que tiene la inversión de recursos en proponer una solución inicial con un criterio que nos debería colocar en un entorno de soluciones parecido al óptimo real, así como su sinergia (o no) con los algoritmos de Hill Climbing y Simulated Annealing.

Predicción sobre los posibles resultados

Teniendo en cuenta como funcionan los algoritmos de Hill Climbing y Simulated Annealing, combinado con nuestras propuestas para generación de estados iniciales, cabría esperar lo siguiente:

Para el uso de Hill Climbing, realizar un esfuerzo inicial para generar un estado inicial favorable considerando distancias y prioridades de los grupos debería acercarnos al óptimo global de todo el espacio de soluciones. Una vez cerca, el algoritmo debería funcionar realizando pocas exploraciones y encontrando un óptimo local rápidamente.

Para el uso de Simulated Annealing, debido a su volatilidad, el mismo esfuerzo inicial resultaría menos provechoso, ya que, al ser un algoritmo mucho más abierto a explorar soluciones aparentemente peores, debería resultar frecuente alejarse de ese óptimo local próximo al óptimo global, proporcionando soluciones de peor calidad pero con mayor rapidez. Para aprovechar esta volatilidad para explorar otras soluciones aparentemente peores, debería resultar interesante no invertir mucho esfuerzo en la generación inicial, dejando que el propio algoritmo haga el trabajo mediante los saltos.

En términos de tamaño del problema, deberíamos ver que, a mayor tamaño, más tiempo se invierte en la generación del estado inicial cuando usamos criterios de distancia y/o prioridad del que se invierte en el uso de los algoritmos, siempre considerando constantes los valores de las variables del algoritmo de Simulated Annealing. Cabría esperar, entonces, que para la generación de estado inicial aleatoria, el tiempo invertido en generar la solución inicial siguiera sin ser una parte significativa del tiempo de ejecución total.

A continuación, se incluyen las reflexiones iniciales acerca de los puntos sugeridos de la práctica:

- Aumento del número de centros (con 1 helicóptero) manteniendo la cantidad de grupos.
Aumentar el número de centros debería reducir el número de estados “hijos”, reduciendo significativamente el tiempo de ejecución del algoritmo Hill Climbing, manteniendo o mejorando la calidad de la solución (tiempo menor). Para el uso de Simulated Annealing, debería ser aproximadamente constante debido a la volatilidad para saltar a soluciones peores.

- Aumento del número de grupos manteniendo el número de centros (5, con 1 helicóptero en cada centro).

El algoritmo de Hill Climbing debería verse afectado al tener que analizar más estados “hijos”, puesto que a mayor cantidad de grupos, mayor cantidad de escenarios “hijos” se pueden generar, haciendo que el tiempo de ejecución del algoritmo pase a tomar mayor protagonismo. El algoritmo de Simulated Annealing debería seguir siendo constante.

En ambos casos la calidad de la solución debería mantenerse, considerando que a más grupos a rescatar, se requerirá un mayor tiempo para el rescate.

- Impacto de aumentar el número de helicópteros por centro, manteniendo el número de centros constantes, comparándolo con aumentar el número de centros con 1 helicóptero en cada uno.

Si bien al inicio podría parecer lo mismo que aumentar el número de centros, el aumento del número de centros debería ser mejor que tener más helicópteros por centro. El motivo es que, a más centros, más puntos alrededor del perímetro del mapa estarían disponibles, lo que reduciría la distancia. Al mantenerse igual el número de helicópteros en ambos casos (1 centro con 5 helicópteros tiene los mismos helicópteros que 5 centros con 1 helicóptero), dispondremos de puntos más cercanos a los grupos, disminuyendo la distancia recorrida total.

- Impacto de la segunda heurística (minimizar tiempo de prioritarios) respecto la primera heurística (minimizar tiempo total)

El hecho de realizar rescates de los grupos prioritarios primero debería ocasionar que el tiempo total de rescate aumente, ya que esta restricción debería implicar la pérdida de posibles optimizaciones relacionadas con la capacidad del helicóptero (hasta 15 personas) o la distancia total del recorrido. Un caso extremo que debería servir para visualizar esta hipótesis sería el siguiente: dado un centro en coordenadas (0,0) con 2 grupos asignados (uno prioritario y uno no prioritario) con los dos grupos en coordenadas (48,49) y el otro en coordenadas (49,48), siendo el prioritario compuesto por 3 miembros y el no prioritario con 12.

Al querer minimizar el tiempo de rescate de prioritarios, realizaríamos un solo viaje a la distancia más alejada del centro, volveríamos y posteriormente realizaríamos otro viaje de la misma distancia, tras penalización por repostaje/descanso del helicóptero, cuando con un solo viaje podríamos haber recogido ambos grupos.

Experimentación y resultados

En primer lugar, se especifica el *hardware* utilizado para estos experimentos. Esta indicación se debe a la medida de tiempos de ejecución, que son variables en función de varios factores, como el procesador, gráfica o sistema operativo.

- CPU: AMD Ryzen 7 5800X 8-Core Processor (3,80 GHz), con arquitectura x86_64.
- RAM: 32 GB con frecuencia 2667 MHz (DDR4).
- GPU: NVIDIA GeForce RTX 4070 (12 GB de VRAM).
- Almacenamiento: 1,82 TB de unidad de estado sólido (SSD).
- Sistema operativo: Windows 11 Home.

Influencia de los operadores

- **Observación Inicial:** Puede que el uso alguno de los dos operadores, Cambio de Orden e Intercambio de Grupos, obtenga un mejor tiempo de ejecución, una mejor calidad de la solución final, o expanda más nodos durante la búsqueda que el otro.
- **Planteamiento:** Escogemos un mismo problema y realizamos ejecuciones utilizando diferentes operadores.
- **Hipótesis:** Los operadores dan lugar a soluciones iguales o hay uno mejor que otro.
- **Método:** Se elige una semilla aleatoria, un problema con un escenario con 5 centros, un helicóptero en cada centro, y 100 grupos a rescatar. Se escoge una estrategia de generación de estado inicial entre las propuestas. Se ha seleccionado la generación inicial aleatoria. Se usará el algoritmo de Hill Climbing y la función heurística (1), que prioriza el tiempo total de rescate de todos los grupos, sin considerar si son grupos prioritarios o no.

Replica	Valor Heurística Cambio Orden	Valor Heurística Intercambio Grupos	Pasos Cambio Orden	Pasos Intercambio Grupos	Tiempo (ms) Cambio Orden	Tiempo (ms) Intercambio Grupos
1	3772.165359462914	4839.348690590552	23.0	2.0	169.0	23.0
2	3792.5485197052903	4441.537972762506	17.0	4.0	74.0	6.0
3	3868.6099576690576	4303.799040112648	20.0	3.0	104.0	4.0
4	3810.8552303334045	4588.729784229203	19.0	5.0	105.0	2.0
5	3666.2371871008445	4552.851802388383	27.0	3.0	118.0	1.0
6	3701.6230315935622	4520.805356805229	25.0	3.0	98.0	1.0
7	3703.1060636657685	4484.880214158989	23.0	5.0	99.0	2.0
8	3701.2189263756295	4472.496743887299	27.0	3.0	116.0	1.0
9	3671.2608354012564	4437.112142430068	26.0	7.0	115.0	2.0
10	3816.7406027684938	4210.2200087329475	22.0	11.0	95.0	4.0
11	3721.754787068638	4608.052788614459	24.0	5.0	96.0	2.0
12	3674.0715945668503	4571.708888766856	20.0	5.0	79.0	2.0
13	3819.554287836194	4400.651546523373	15.0	5.0	60.0	2.0
14	3684.97368421966	4506.109627398479	23.0	4.0	96.0	2.0
15	3741.824193370998	4421.571274115029	25.0	5.0	94.0	2.0
16	3728.543116136811	4652.054088171666	25.0	3.0	112.0	1.0
17	3738.869667974761	4642.186984036391	22.0	5.0	101.0	2.0
18	3649.956246418373	4660.162429692086	20.0	4.0	89.0	1.0
19	3782.180298776128	4485.713649112833	23.0	5.0	87.0	2.0
20	3670.2473462536213	4696.959270999105	27.0	5.0	110.0	2.0
21	3832.4254522442006	4589.573151169436	26.0	3.0	106.0	2.0
22	3741.2968005465546	4461.568720171814	24.0	3.0	84.0	2.0
23	3753.803475551303	4412.1323945601325	24.0	3.0	87.0	1.0
24	3672.411425211608	4492.65321890686	24.0	2.0	93.0	0.0
25	3857.943784658404	4591.549618473684	23.0	1.0	91.0	0.0

Figura 1: Valor de la solución, número de nodos expandidos y tiempo de ejecución en milisegundos.

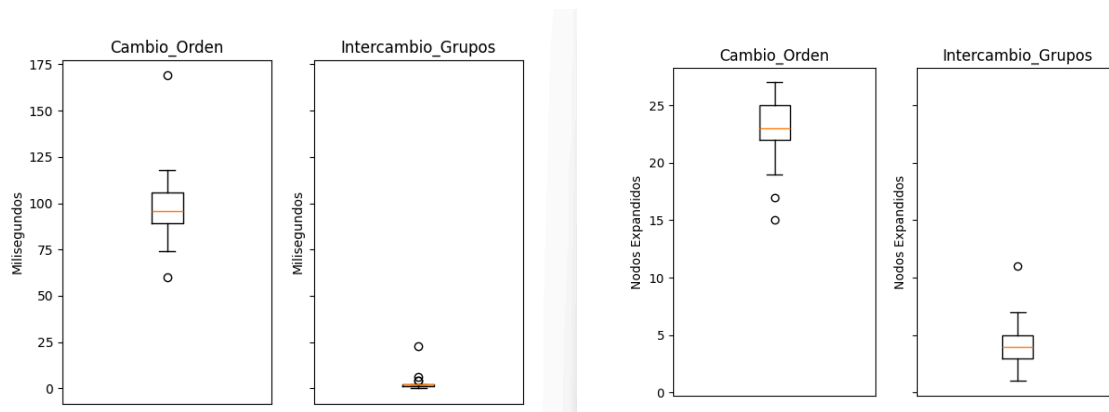


Figura 2: Distribución del tiempo de ejecución y de los nodos expandidos según el operador utilizado.

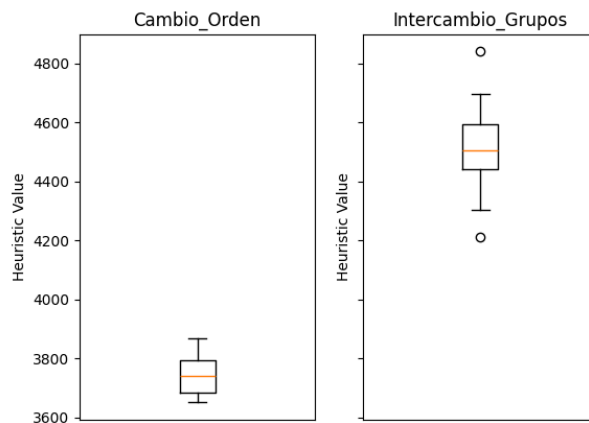


Figura 3. Distribución de los valores de la solución diferenciada según el operador utilizado.

Los resultados obtenidos muestran que usar el operador *Cambio_Orden* resulta en un tiempo de ejecución mayor, así como una mayor expansión de nodos. Ambas mediciones están relacionadas, pues a mayor búsqueda dentro del espacio de soluciones, mayor será el tiempo de ejecución del algoritmo Hill Climbing. Pero, por otro lado, el valor de la solución es más bajo, indicando que el operador tiende a guiarnos hacia soluciones de mayor calidad. El operador *Intercambio_Grupos* nos ofrece una expansión de nodos considerablemente menor y, por ello, un tiempo de ejecución inferior. No obstante, el valor de la solución final tiende a ser mucho mayor; es decir, de menor calidad. Esto se debe a que el uso de este operador limita el espacio de búsqueda, por lo que hallar una solución de buena calidad es más complicado.

El uso del operador Cambio Orden resulta en una mejor calidad de la solución final a costa de una ejecución más lenta, pero también una búsqueda relativamente más exhaustiva. Para los posteriores experimentos, se considerará el Cambio Orden como el operador a utilizar, debido a que se prioriza minimizar el tiempo de rescate de los grupos por encima del tiempo de ejecución.

Influencia de la solución inicial

- **Observación Inicial:** La generación de la solución inicial puede afectar a la calidad de la solución, tiempo de ejecución y el número de nodos expandidos en la búsqueda.

- **Planteamiento:** Escogemos diferentes métodos de generación de solución inicial, entre los cuatro planteados, y observamos sus soluciones.
- **Hipótesis:** Todos los generadores de estados iniciales son iguales (H_0) o hay algunos mejores que otros.
- **Método:** A partir del mejor operador obtenido en el experimento anterior, se escogen problemas de las mismas características, 5 centros, un helicóptero por centro, y 100 grupos a rescatar. Se realizan 25 iteraciones por cada una de las generaciones de solución inicial, manteniendo todas la misma semilla. Se utiliza Hill Climbing y la función heurística (1).

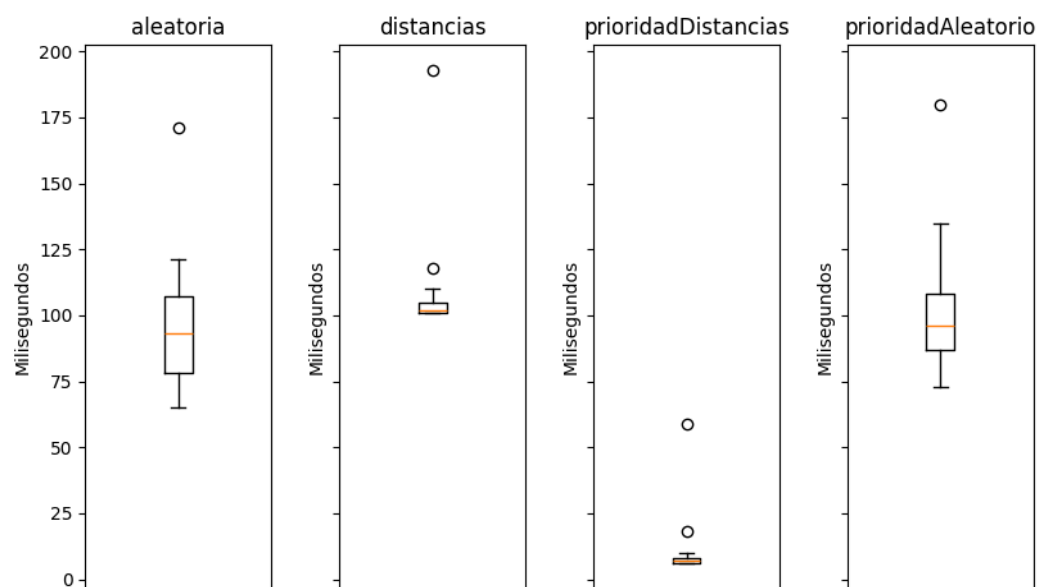


Figura 4. Distribución de tiempos de ejecución (en milisegundos) de los diferentes generadores de estado inicial.

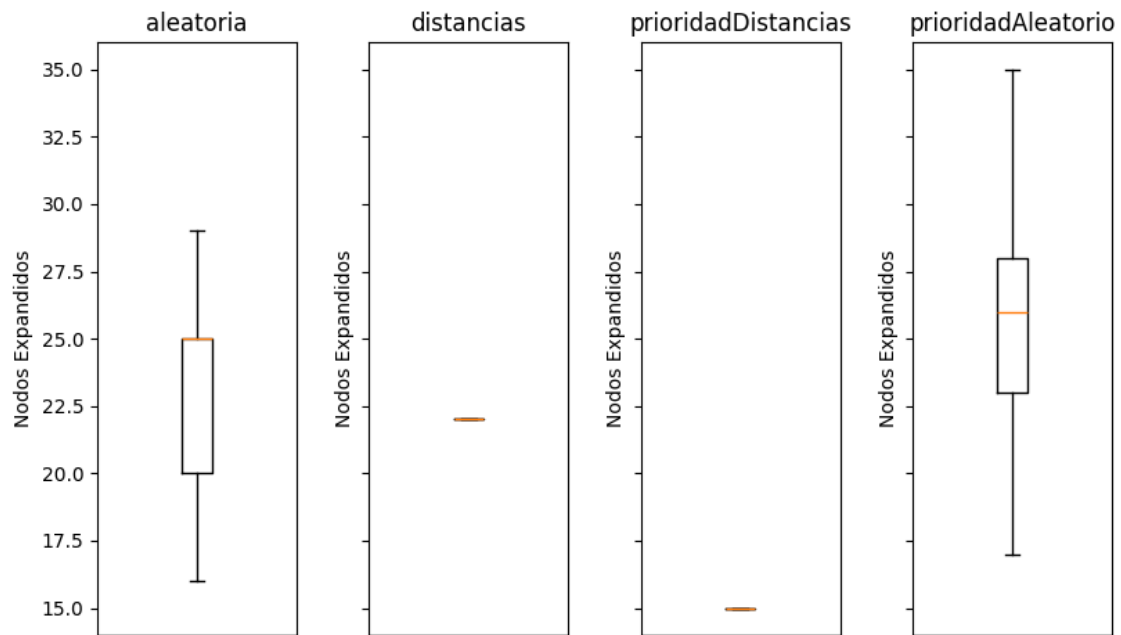


Figura 5. Distribución de los nodos expandidos hasta encontrar una solución óptima según el generador utilizado.

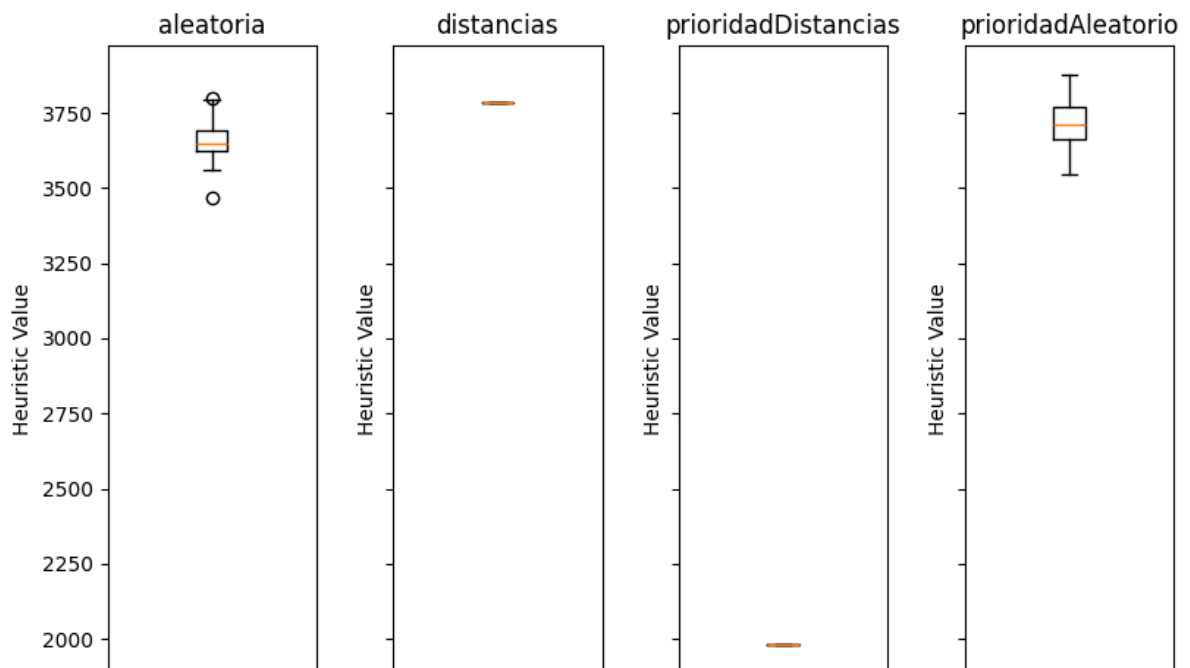


Figura 6. Distribución del valor heurístico de la solución encontrada según el generador de solución inicial empleado.

	Aleatorio	Distancias	PrioridadDistancias	PrioridadAleatorio
Nodos	23.0 (3.83)	22.0 (0.0)	15.0 (0.0)	25.92 (4.6)
Tiempo	96.24 (21.95)	107.48 (17.9)	9.44 (10.39)	101.76 (22.96)

Figura 7. Media y desviación típica de los nodos expandidos y el tiempo de ejecución según el generador utilizado, formato Media (Desviación Típica).

Los resultados obtenidos nos muestran diferencias significativas en las diferentes ejecuciones, según el generador de solución inicial escogido. La generación aleatoria, nombrada como *AleatorioTotal* en el código, resulta en un tiempo de ejecución distribuido entre 70 y 125 milisegundos. La expansión de nodos es variable, dependiendo de la ejecución. El valor máximo de nodos explorados tiene una distribución amplia, indicándonos que la profundidad con la que se explora el espacio de estados se ve afectada por la aleatoriedad de la ejecución. La calidad de la solución es significativamente peor.

La generación por *distancias* nos da una desviación típica muy próxima a 0 respecto al número de nodos expandidos, debido a que la distribución está concentrada entre valores muy próximos entre sí. El valor heurístico nos da un valor elevado, cercano a 3750 minutos. El tiempo de ejecución es similar al tiempo que se obtiene al usar *AleatorioTotal*.

La generación por *PrioridadDistancias* nos da el menor tiempo de ejecución de todas las cuatro generaciones, teniendo una media de solo 9.44 milisegundos. Se expanden pocos nodos durante la búsqueda (desviación típica próxima a 0), que, recordemos, está relacionado con el tiempo de ejecución. No obstante, a diferencia de otros experimentos vistos anteriormente, también obtenemos un valor heurístico bajo.

La generación *PrioridadAleatorio* nos da un tiempo de ejecución similar al tiempo que obtenemos con la generación aleatoria. Obtenemos la distribución más amplia de las cuatro generaciones, pero también una mala calidad de la solución.

Los resultados obtenidos reflejan que la calidad de la solución es similar en todos los casos, a excepción del método de generación con colas de prioridad y distancias. Adicionalmente, este método resulta ser el más rápido, pero que también expande menor número de nodos. Las conclusiones obtenidas nos hacen pensar que el método *PrioridadDistancias* impone un estado inicial cercano a la solución, por lo que el algoritmo de Hill Climbing encuentra rápidamente una solución. Cabe recordar que el método Prioridad Distancias afronta el

problema tratando dos elementos cruciales para la resolución de este: prioridad de los grupos a rescatar, y la asignación de un grupo al centro más cercano.

Por lo tanto, se considerará para los próximos experimentos el método *PrioridadDistancias* como el más adecuado para obtener una solución de mejor calidad.

Influencia de Parámetros de Simulated Annealing

- **Observación Inicial:** Los parámetros del algoritmo de búsqueda Simulated Annealing pueden influir considerablemente en los resultados de las ejecuciones.
- **Planteamiento:** Escogemos diferentes parámetros iteraciones y k realizando pruebas donde se fijan los demás parámetros del Simulated Annealing.
- **Hipótesis:** La calidad de las soluciones encontradas depende de las variables iteraciones y k del Simulated Annealing.
- **Método:** Se repiten las mismas características del problema, 5 centros, un helicóptero por centro, y 100 grupos a rescatar. A partir de las conclusiones obtenidas anteriormente, mejor operador y mejor generador de estado inicial, se ejecutan dos pruebas. La primera, determina el número de iteraciones a realizar. Para ello, se incrementa la variable steps de 200 en 200 y se mide la calidad de la solución final con la función heurística (1). La segunda prueba mide la influencia del parámetro K en Simulated Annealing. Manteniendo el valor determinado de steps, se incrementa de 1 en 1 el valor de K.

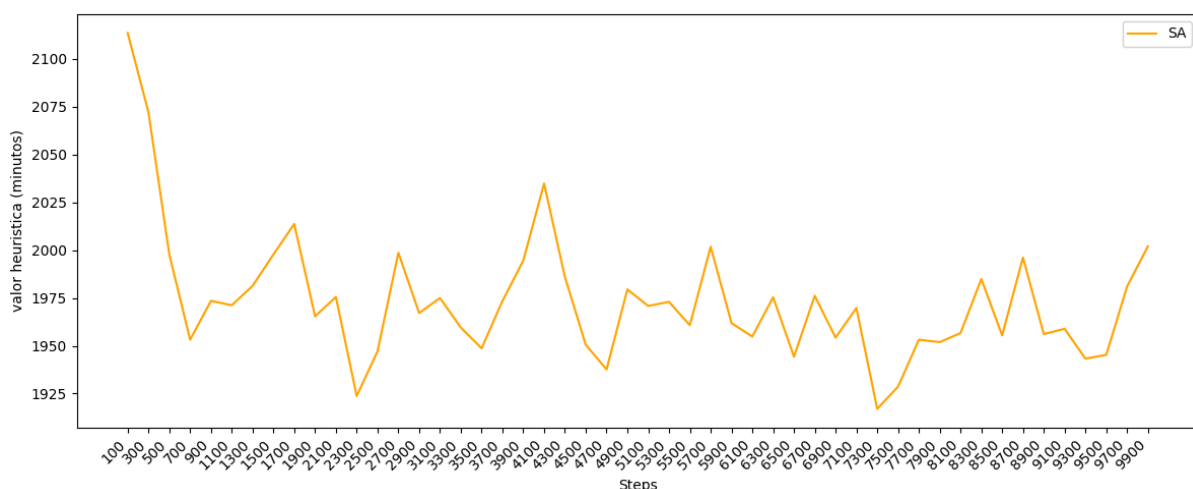


Figura 8. Evolución de la calidad de la solución encontrada según el número de iteraciones (steps) realizado en el Simulated Annealing.

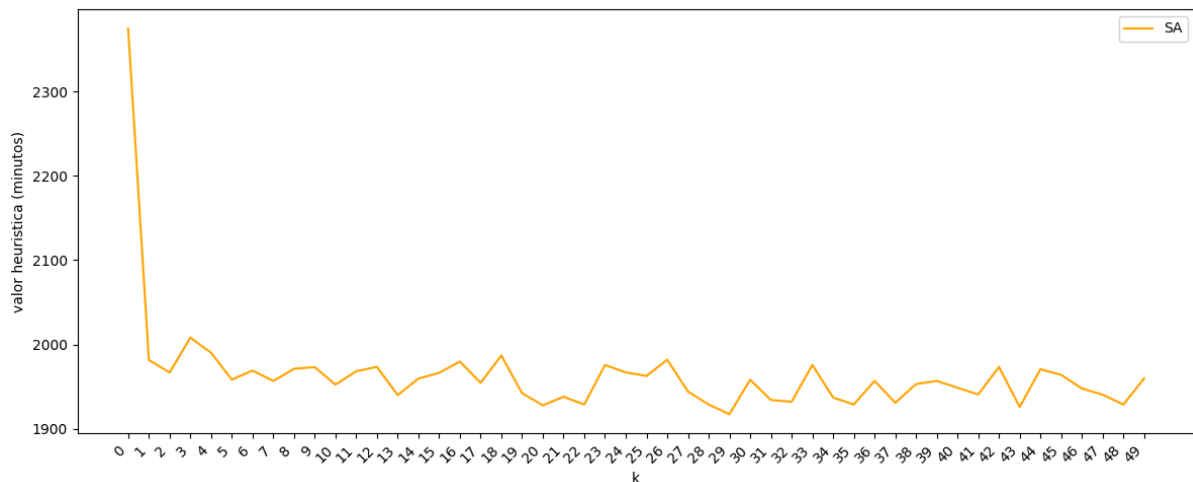


Figura 9. Evolución de la heurística de la solución encontrada según el número de iteraciones (steps) realizado en el Simulated Annealing.

En ambas gráficas se puede observar que el coste de la solución varía considerablemente si cambiamos el valor de iteraciones máximas que realiza Simulated Annealing. Se observa que se obtiene la mejor calidad con iteraciones = 7500. Fijamos ese parámetro para la segunda prueba. En la figura 9 se puede apreciar que el valor de k también influye en la solución inicial, aunque no de una forma tan significativa como el número de iteraciones. Varios mínimos se encuentran cercanos a un mismo valor, por lo que se escoge k = 48. Para los próximos experimentos, se utilizarán los parámetros iteraciones = 7500 y k = 48.

Influencia del tamaño del problema con relación 5:100

- **Observación Inicial:** El tamaño del problema podría influir en los tiempos de ejecución manteniendo una proporción fija entre la cantidad de centros y la cantidad de grupos a rescatar.
- **Planteamiento:** Se ejecuta Hill Climbing y Simulated Annealing variando el tamaño del problema y observamos su tiempo de ejecución.
- **Hipótesis:** Simulated Annealing obtendrá tiempos de ejecución menores a Hill Climbing, que tendrá tiempos de ejecución por lo menos exponenciales.
- **Método:** Dado el escenario de los apartados anteriores, se estudia cómo evoluciona el tiempo de ejecución variando los parámetros Centros y Grupos del problema. Para ello, se ejecutaron Hill Climbing y Simulated Annealing con los mismos valores

que los determinados en los anteriores experimentos. Se ejecutan un total de 16 iteraciones donde en cada iteración, el número de centros aumenta en 5, siempre manteniendo una relación 5:100 con el número de grupos a rescatar.

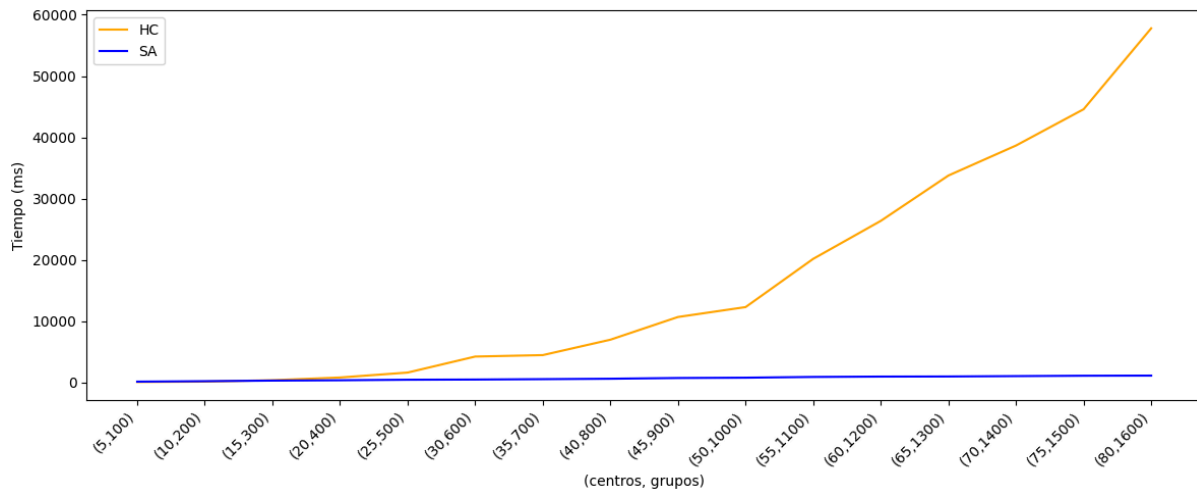


Figura 10. Función del tiempo en milisegundos de Hill Climbing y Simulated Annealing según el tamaño del problema.

La evolución del tiempo de ejecución muestra que, aun incrementando el tamaño del problema Simulated Annealing alcanza unos tiempos de ejecución muy pequeños en comparación a Hill Climbing, y que la cantidad de (*centros, grupos*) no parece influir de una forma considerable en el rango de tamaños utilizado. Por otro lado, se observa que Hill Climbing se ve afectado por la cantidad de centros y grupos a rescatar. La tendencia marcada por la función amarilla refleja un crecimiento altamente creciente.

Influencia del tamaño del problema con valores (+5, +50)

- **Observación Inicial:** El tamaño del problema podría influir en los tiempos de ejecución incrementando únicamente la cantidad de centros o la cantidad de grupos a rescatar.
- **Planteamiento:** Se ejecuta Hill Climbing variando el tamaño del problema y observamos su tiempo de ejecución.
- **Hipótesis:** El incremento del número de grupos a rescatar aumentará el tiempo de ejecución, mientras que el incremento de centros lo disminuirá.

- **Método:** Dado el escenario de los apartados anteriores, se estudia cómo evoluciona el tiempo de ejecución variando los parámetros Centros y Grupos por separado. Para ello, se ejecuta Hill Climbing con los mismos valores que los determinados en los anteriores experimentos en dos pruebas diferentes. En la primera prueba, se ejecutan 16 iteraciones donde se incrementa el tamaño de grupos a rescatar en 50. En la segunda prueba, se ejecutan 16 iteraciones incrementando la cantidad de centros disponibles en 5.

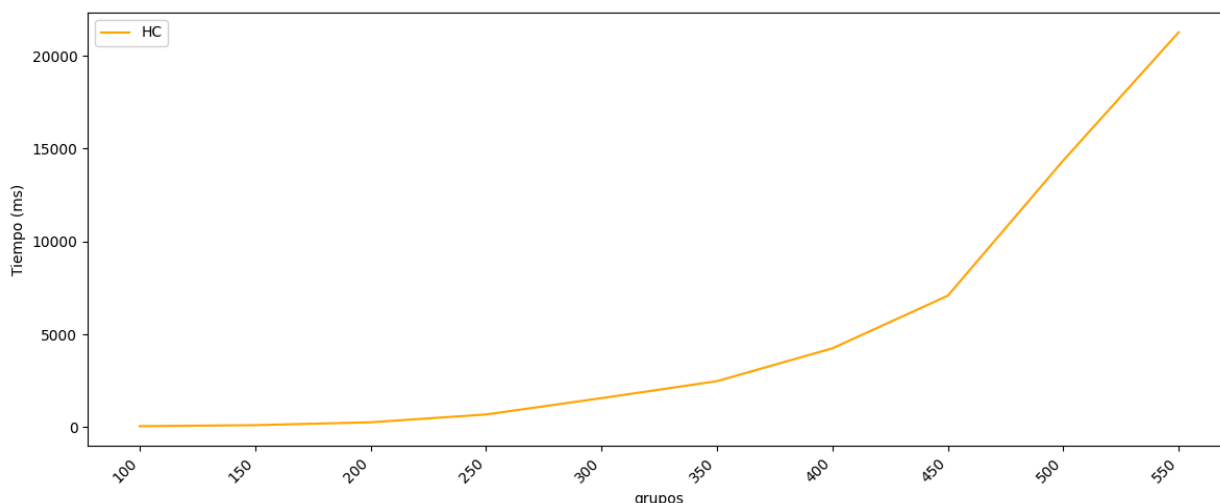


Figura 11. Función del tiempo en milisegundos de Hill Climbing según el número de grupos a rescatar.

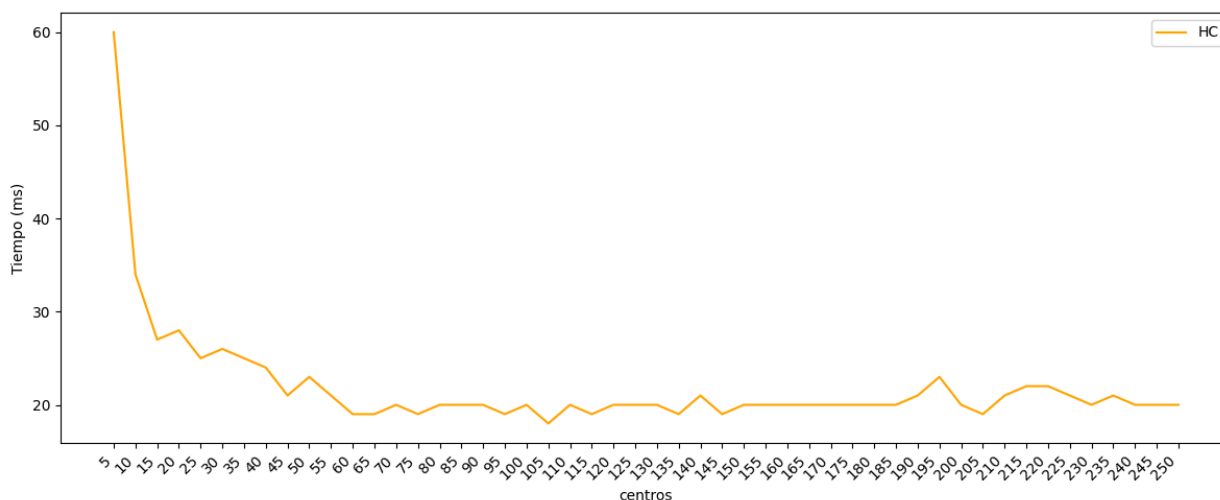


Figura 12. Función del tiempo en milisegundos de Hill Climbing según el número de centros disponibles.

Las dos funciones nos muestran que al aumentar el tamaño de los grupos, el tiempo de ejecución de Hill Climbing aumenta, mientras que al incrementar el número de centros, el tiempo disminuye. Si contrastamos este experimento con los resultados obtenidos en el

anterior, observamos que el número de centros es un factor restante en el tiempo de ejecución. Por ejemplo, si comparamos el tiempo de ejecución con 500 centros, observamos que en el anterior experimento el tiempo de ejecución es menor a 5000 milisegundos; pero en este experimento el tiempo aumenta hasta 15000 aproximadamente. Esto nos hace pensar que si no se aumenta el número de centros, el tiempo de ejecución crece de una forma mucho más rápida. Cabe destacar que en todo momento estamos utilizando solo un helicóptero por centro, algo que estudiaremos en el siguiente experimento.

Influencia del incremento de números de helicópteros disponibles

- **Observación Inicial:** La calidad de la solución y el tiempo de ejecución podrían variar respecto a la cantidad de helicópteros disponibles sin modificar la cantidad de centros ni heridos.
- **Planteamiento:** Se estudia la evolución temporal de Hill Climbing según el número de helicópteros disponibles por centro.
- **Hipótesis:** Un exceso de helicópteros podría llevar a peores soluciones.
- **Método:** Dado el escenario de los apartados anteriores, se estudia cómo evoluciona el tiempo de ejecución variando la cantidad de helicópteros en cada centro. Utilizando Hill Climbing, se ejecutan 50 iteraciones, donde en cada una de ellas se incrementa el valor de H en 1. Después, se mide tanto el tiempo de ejecución como el valor de la heurística en la solución final.

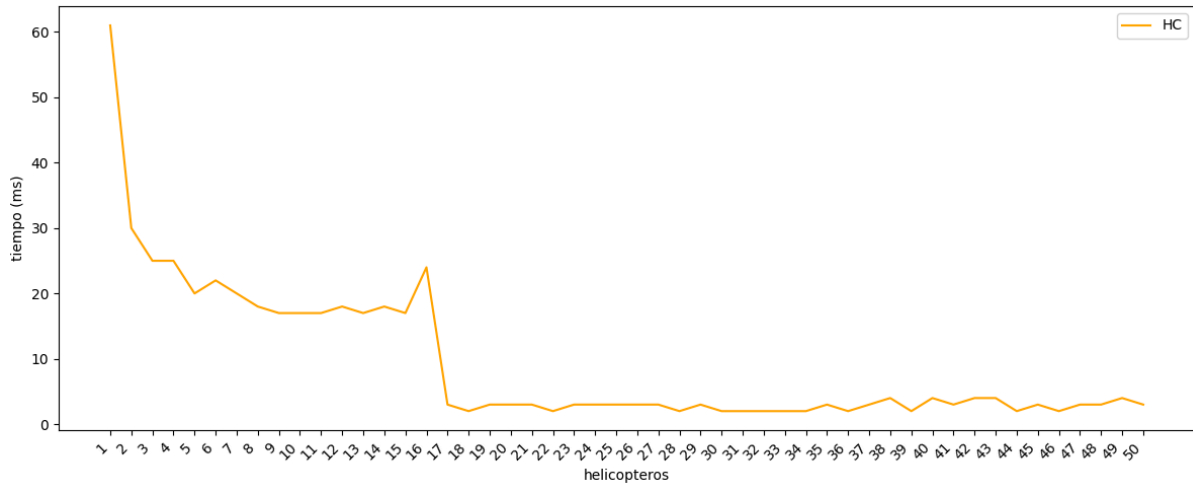


Figura 13. Función del tiempo en milisegundos de Hill Climbing según el número de helicópteros por centro.

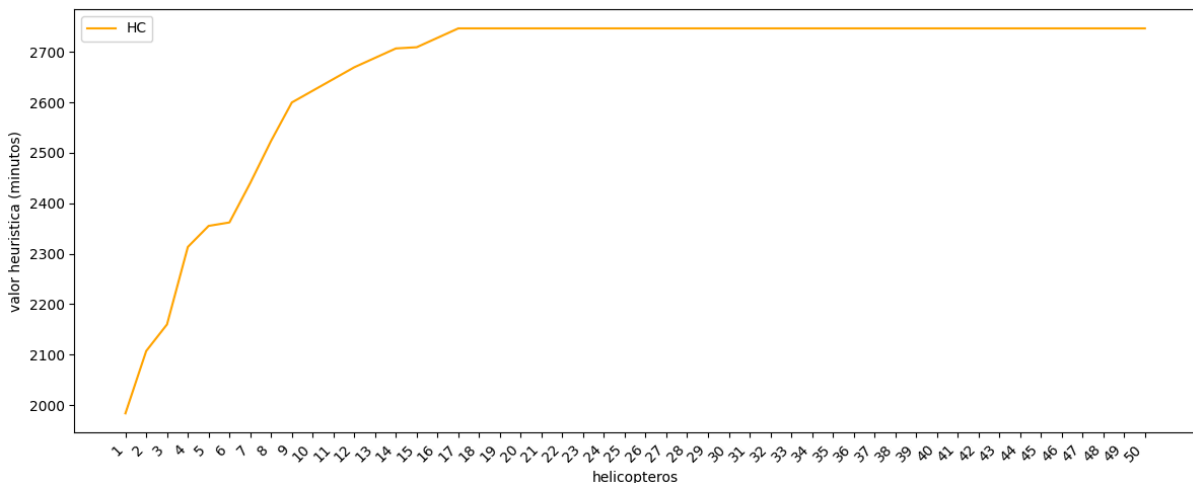


Figura 14. Función del heurístico de la solución de Hill Climbing según el número de helicópteros por centro.

De una forma similar al número de centros, el incremento del número de helicópteros disminuye el tiempo de ejecución del algoritmo de Hill Climbing. Sin embargo, la figura 14 muestra que el valor de la heurística aumenta, llevándonos a una peor calidad de la solución. Cabe recordar que el número de centros y grupos no aumenta en este experimento: hay un total de 5 centros y 100 grupos. Por tanto, al aumentar el número de helicópteros de manera excesiva, obtendremos escenarios donde muchos helicópteros obtengan una carga mal distribuida. Es decir, situaciones donde algunos helicópteros deben de rescatar a un gran número de grupos mientras que otros tienen una carga nula. Es más, la generación *PrioridadDistancias* escogida anteriormente podría contribuir a este problema, puesto que asignaría grupos al centro más cercano, mientras que centros lejanos no recibirán ninguna asignación.

Diferencia entre las dos heurísticas

- **Observación Inicial:** Las dos funciones heurísticas, añadiendo ponderaciones a la heurística dos para priorizar el rescate de grupos heridos, pueden resultar en diferentes valores de solución.
- **Planteamiento:** Se estudia la evolución de la heurística de Hill Climbing con una ponderación donde se prioriza el rescate de grupos prioritarios al incrementar el tamaño del problema.
- **Hipótesis:** Obtendremos valores más elevados en la heurística que prioriza el rescate de grupos heridos.
- **Método:** Dado el escenario de los apartados anteriores, se estudia cómo evoluciona la calidad de la solución según el uso de la heurística *tiempo_total* y la heurística *prioridades*. Se harán 16 iteraciones donde el tamaño del problema variará con la relación 5:100 hablada anteriormente. Se realizan comparaciones entre Hill Climbing con la heurística uno y la heurística dos..

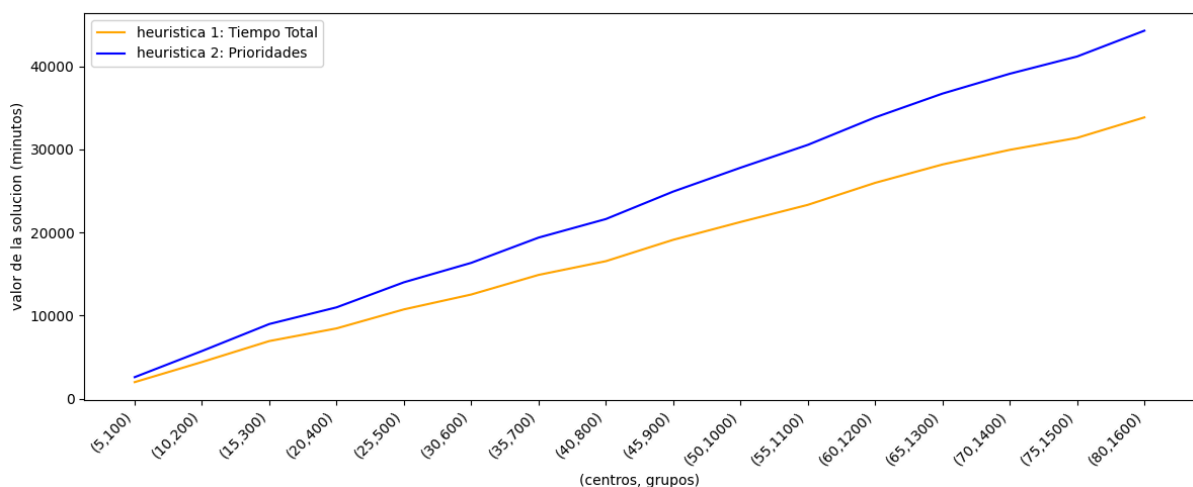


Figura 15. Evolución del valor heurístico de la solución según el tamaño del problema doblando el peso al rescate de grupos prioritarios y utilizando Hill Climbing.

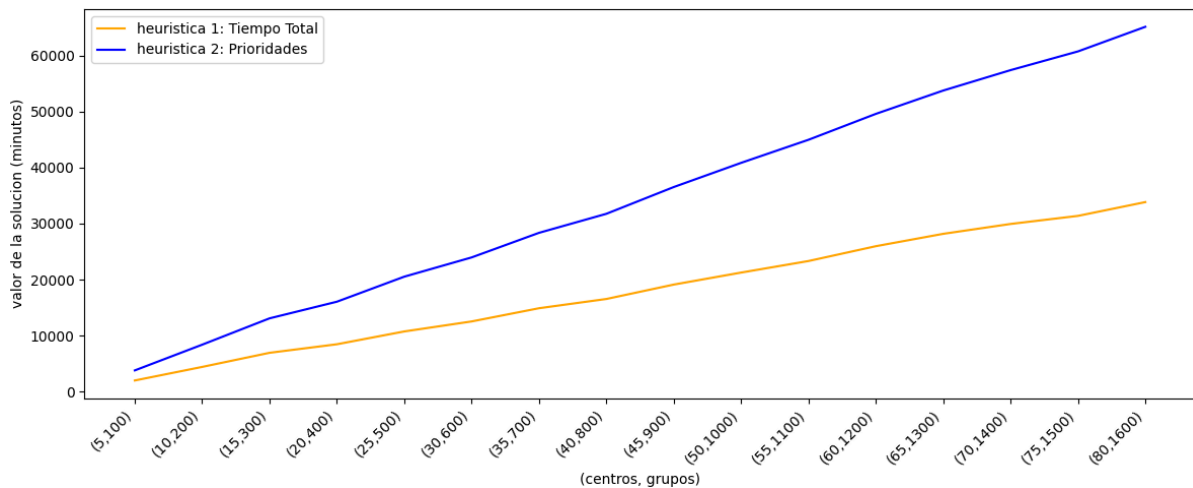


Figura 16. Evolución del valor heurístico de la solución según el tamaño del problema, cuadruplicando el peso al rescate de grupos prioritarios y utilizando Hill Climbing.

Los resultados obtenidos reflejan el valor heurístico de la solución encontrada. Al añadir una ponderación doble al segundo criterio, se observa cómo el tiempo necesitado para rescatar a todos los grupos (valor de la solución en minutos) es superior respecto al tiempo con el primer criterio. Al aumentar la ponderación a cuatro sobre el segundo criterio, la diferencia del valor de las soluciones encontradas es mayor. Por tanto, se puede observar que priorizar el segundo criterio aumenta el tiempo que tardamos en rescatar a todos los grupos.

Conclusiones

Si contrastamos los resultados obtenidos en la fase de experimentación con nuestras predicciones sobre los resultados, podemos concluir que gran parte de nuestras hipótesis se cumplen.

Una generación de solución inicial elaborada influye positivamente en el hallazgo de un óptimo local. Como se ha visto en el experimento Influencia de la Solución Inicial, la generación Prioridad con Distancias, dando preferencia a los grupos prioritarios y a asignar grupos al centro más cercano, ayuda a que el algoritmo Hill Climbing encuentre una solución expandiendo un número bajo de nodos y un bajo tiempo de ejecución, pero sin perder calidad de la solución encontrada.

Como se ha podido observar en el experimento Influencia de los parámetros de Simulated Annealing, la calidad de la solución puede variar significativamente aun utilizando la generación de estado inicial Prioridad + Distancias. La búsqueda de un óptimo local con Simulated Annealing no se ve tan afectada por la solución inicial, sino por los parámetros utilizados.

El tamaño del problema influye significativamente en el tiempo de ejecución para Hill Climbing. Al aumentar el número de grupos a rescatar, incrementa también el tiempo de ejecución; sin embargo, el número de centros es un factor restante en el tiempo de ejecución. Por otro lado, se ha visto que tener un exceso de helicópteros disminuye el tiempo de ejecución, pero a costa de obtener un óptimo local de peor calidad. Por ello, es necesario dar importancia a un correcto equilibrio entre centros, grupos, y que no haya un exceso de helicópteros respecto al número de grupos del escenario.

Dar importancia a diferentes criterios afecta al valor de la solución encontrada. Si se da más ponderación a los grupos prioritarios, es decir, el criterio segundo, se obtiene un valor heurístico mayor. En contraste, si se prioriza rescatar a todos los grupos, heridos o no, se obtiene un valor heurístico menor. Por tanto, se observa que priorizar a los heridos implica una tardanza mayor a la hora de rescatar a todos los grupos del escenario.

Competencia transversal: Trabajo de Innovación

Las tres personas que han elaborado este trabajo también forman el grupo del Trabajo de Innovación. Trata sobre Neuro-sama, una *VTuber* que no está protagonizada por una persona, sino por una inteligencia artificial. Además, es la *VTuber* con más seguidores de Twitch.

A estas alturas, se ha recopilado información sobre el origen y la evolución de Neuro-sama en la emisión de contenido en internet, hallando también algunos datos sobre cómo podría estar elaborada. Además, se ha empezado a redactar la documentación, incluyendo un resumen de dicha trayectoria virtual y un análisis de los pros y los contras de su existencia en redes sociales.

El reparto del trabajo ha sido el siguiente:

- Jorge de la Fuente: búsqueda de información sobre la trayectoria por internet de Neuro-sama.
- Pol García Vernet: Búsqueda de información sobre las herramientas usadas para el desarrollo de Neuro-Sama.
- Roger Catalan: proponente del tema de este proyecto. Búsqueda de información sobre las diferentes áreas relacionadas con el proyecto Neuro-sama.

A continuación, se exponen los enlaces utilizados hasta el momento, incluyendo su fecha de consulta y un comentario sobre cada fuente.

- <https://virtualyoutuber.fandom.com/wiki/Neuro-sama>. Consultado el 21/02/2026. A pesar de tratarse de wikis (es decir, páginas de información creadas por el esfuerzo colaborativo de personas anónimas alrededor del mundo), los artículos de Fandom, por experiencia, son muy buenos. Grupos de personas con una pasión en común se dedican a fijarse hasta el más mínimo detalle en algún producto de entretenimiento, ya sea la historia, secretos y listas de elementos a completar en un videojuego, sagas de libros o películas. En nuestro caso, había hasta descripciones detalladas, con fechas incluidas, sobre los diversos cambios estéticos que ha tenido el avatar de Neuro-sama.
- <https://neurosama.fandom.com/wiki/Neuro-sama>. Consultado el 02/03/2026. Los mismos argumentos utilizados para la fuente anterior, se puede atribuir a este. Las

páginas son distintas porque se tratan de artículos con el mismo tema situados en bloques temáticos distintos: Fandom de *Virtual YouTuber* y Fandom de Neuro-sama. Las informaciones de esta fuente y la anterior son coherentes entre sí y se complementan.

- https://virtualyoutuber.fandom.com/wiki/Evil_Neuro. Consultado el 02/03/2026. El argumento utilizado para las fuentes anteriores es el mismo. Aquí se agrega información sobre Evil Neuro, otra inteligencia artificial que emite en el mismo canal de Twitch que Neuro-sama (Vedal987). Muy útil para tener una comparativa sobre sus diferentes “personalidades”, enriqueciendo aún más nuestra búsqueda.
- <https://en.wikipedia.org/wiki/Neuro-sama>. Consultado el 21/02/2026. Esta página se complementa con las dos fuentes anteriores. Añadida, porque ofrece cifras de suscripciones y *bíts* por varias temporadas, indicando las cifras exactas obtenidas cuando batió récords.
- <https://www.youtube.com/watch?v=wZ0osmPIsY&t=1s>. Consultado el 21/02/2026. Fuente utilizada cuya información coincide con la información ya buscada, lo que nos lleva a deducir que se trata de una fuente fiable.
- <https://www.xataka.com/robotica-e-ia/neuro-sama-ia-que-ha-destronado-a-todos-streamers-humanos-twitch-porque-no-necesita-comer-dormir>. Consultado el 21/02/2026. Se hace eco de la fama y rentabilidad de Neuro-sama en Twitch, haciendo una mención de los nombres de los lenguajes de programación y motores empleados. Esto último encaja con información de la primera fuente. Sin embargo, dota de varias imprecisiones. El mismo título indica que el canal de Twitch emite las 24 horas del día, lo cual no es cierto.
- <https://www.youtube.com/watch?v=XLtCHZt77qg&t=1038s>. Consultado el 01/03/2026. Vídeo del canal de YouTube *Kraul Express*. Tras la fama de Neuro-sama, muchos canales se dedicaron a recoger extractos de momentos curiosos de los directos. En este se ve como la inteligencia artificial empieza a fallar en pleno directo. Ver directos de varias horas es inasumible para unos estudiantes universitarios. Por tanto, la existencia de estos canales de recopilación han simplificado mucho la investigación.
- <https://www.youtube.com/watch?v=cHTc58Blpes>. Consultado el 01/03/2026. Vídeo del canal de YouTube *NeuroCrumbs*, que es otro canal que se dedica a la recopilación de extractos de Neuro-sama, lo que se trata de un magnífico aporte a la investigación. En este se produce un momento cómico donde Neuro-sama y su creador conversan, produciéndose momentos cómicos.
- <https://www.youtube.com/@Neurosama>. Consultado el 01/03/2026. Canal de YouTube oficial de Neuro-sama, encargado de promocionar dicho contenido. A

diferencia de los dos canales anteriores, este se encarga de un canal perteneciente al mismo creador de la inteligencia artificial.

En contraposición, se ha de indicar que se han encontrado dificultades en la búsqueda de información sobre la creación de dicha, debido a que estamos hablando de que es creada por un grupo reducido de personas en concreto, y no por una gran corporación. Además, se suma a que no es de código abierto.